

Você é um engenheiro de software sênior. Crie um **Plano de Implementação detalhado** para um novo endpoint **POST /prospecting-account** dentro do projeto **prospecting-service**, voltado para a prospecção automatizada de contadores. O plano deve incluir arquitetura, fluxo de processamento, modelagem de dados, integrações, tratamento de erros, scripts de automação e considerações de operação.

Contexto do projeto

- Projeto: **prospecting-service**
- Servidor: Linux Ubuntu
- Integração externa: **Z-API** (validação de números e envio de mensagens via WhatsApp)
- Banco de Dados: ragdb

Fluxo de funcionamento

1. Leitura dos dados

- Buscar registros da tabela **prospecting_data_source**, lendo os campos: **cnpj**, **email**, **razao_social**, **telefone1**, **telefone2**.

2. Validação de telefone (WhatsApp)

- Para cada registro, validar se **telefone1** (quando tiver valor) é um número existente no WhatsApp.
- Se **telefone1** não tiver valor ou não for cadastrado no WhatsApp, repetir o mesmo procedimento para **telefone2**.
- **Como validar via Z-API:** usar o endpoint **phone-exists** para verificar se o número tem WhatsApp, sem precisar enviar mensagem:
 - **GET**
https://api.z-api.io/instances/SUA_INSTANCIA/token/SEU_TOKEN/phone-exists/5511999999999
 - Basta substituir o número no final da URL pelo número a ser verificado.
 - **Obrigatório:** enviar o **Client-Token** no header.
 - **Formato do número:** completo, com DDI + DDD + número (ex.: **551129512299**).
 - A resposta indica se o número existe ou não no WhatsApp.
- Se um dos dois telefones for válido:
 - Gravar em uma nova tabela **prospecting_processed** os campos: **cnpj**, **email**, **razao_social**, **telefone_valido**, **data_contato**, **data_resposta**, **status**.
 - Atualizar o campo **status** (a ser criado) na tabela **prospecting_data_source** com o valor do telefone válido.
- Se nenhum dos dois telefones for válido:
 - Incluir a mensagem '**Nenhum telefone válido**' no campo **status** da tabela **prospecting_data_source**.

3. Envio da mensagem

Após incluir o registro, a aplicação deve:

- Sortear um número inteiro **X** entre 1 e 5.
- Buscar a mensagem correspondente no arquivo `prospecting-service\src\main\resources\messages\msg_contador_X.txt` (formato do nome do arquivo).
- Usar a mensagem do arquivo e o **telefone_valido** para enviar a mensagem via **Z-API**.
- Se o envio ocorrer sem erro:
 - Atualizar o registro correspondente na tabela `prospecting_processed`, preenchendo **data_contato** e **status** com o valor '**Contato Inicial**'.
- Incluir um registro na nova tabela `prospecting_audit` com os campos: **data_envio**, **cnpj**, **status** (valores: **Ok** ou **Error**), **log** (mensagem de erro, caso exista).

4. Intervalo entre registros

- Após o envio da mensagem e o registro na tabela de auditoria, sortear um número inteiro **T** entre 5 e 22.
- Utilizar **T** como tempo de espera em **minutos** antes de ler o próximo registro da tabela `prospecting_data_source` e repetir o processo.

Restrições de horário de funcionamento

- As mensagens devem ser enviadas **apenas** de **segunda-feira a sexta-feira**, entre **08:00 e 16:40**.

Automação (scripts Linux/Ubuntu)

Criar os comandos e/ou scripts necessários para que o endpoint seja:

- Disparado toda **segunda-feira às 08:00**.
- Finalizado às **18:00 de sexta-feira**.
- Verificado **a cada hora** se o endpoint está executando (via tabela `prospecting_audit`).
- Caso não esteja executando, enviar uma mensagem de alerta com o subject "**Prospecção de Contadores parada**" para os emails: `audalio.devops@gmail.com`, `audalio@gmail.com` e `queirozrei@gmail.com`.

O que o plano deve entregar

1. Modelagem das tabelas `prospecting_processed`, `prospecting_audit` e alteração na `prospecting_data_source` (DDL/SQL).
2. Estrutura do endpoint (controllers, services, repositórios) e responsabilidades de cada camada.
3. Fluxograma/lógica do processamento principal, incluindo tratamento de erros e retries.
4. Integração com a Z-API: validação de número via `phone-exists` (com `Client-Token` no header e número no formato DDI + DDD) e envio de mensagem.
5. Gerenciamento dos arquivos de mensagem `msg_contador_X.txt`.

6. Scripts de automação (cron/systemd) para agendamento, encerramento e monitoramento com envio de alerta por email.
 7. Considerações de concorrência, robustez, logging e configuração (variáveis de ambiente, como instância, token e client-token da Z-API).
-